

Stage-1 Calibration: Test Procedure and Robot Program

Running the capture session and what the robot program must do



The session produces data plus an exact table of poses



186

robot poses, 5 frames each



~2 h

robot time with fixture changes and checks



0.1 mm

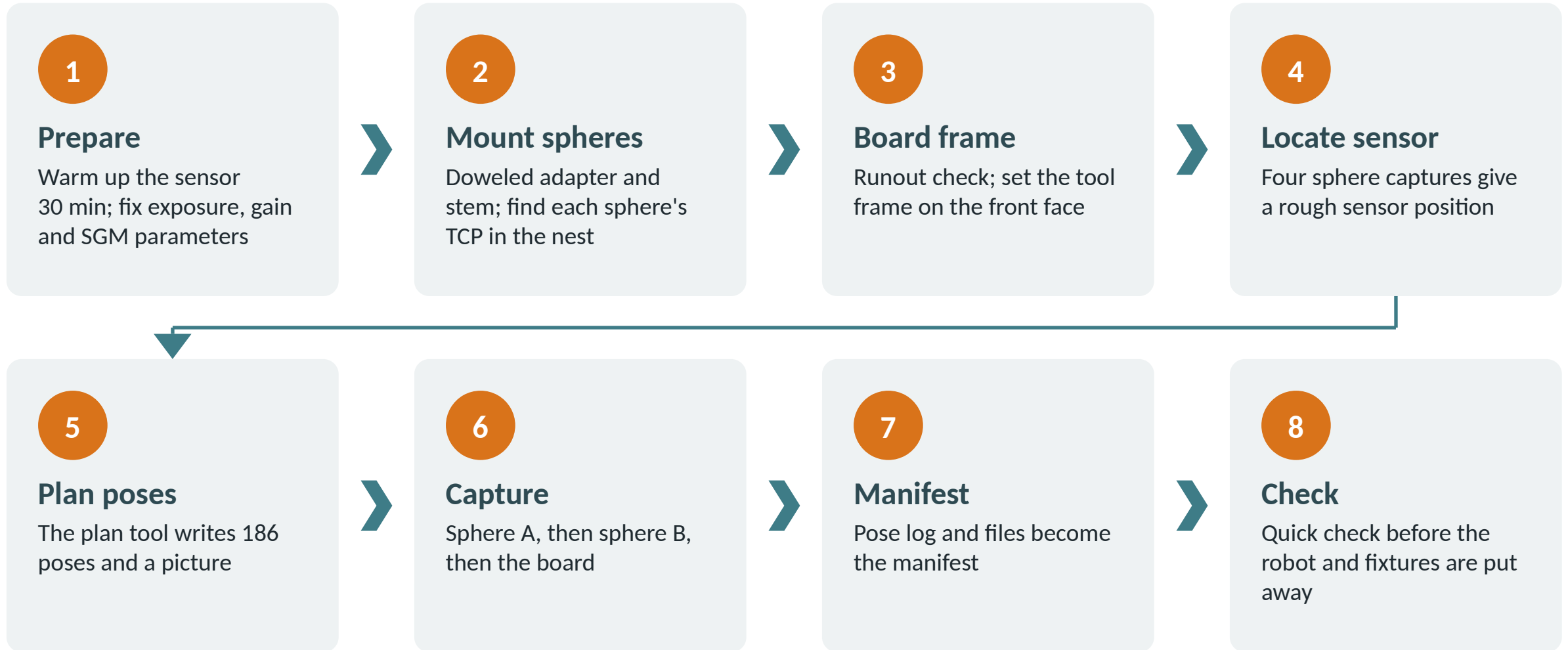
accuracy target at 500 mm, scaling with distance



If the pose table is wrong, the calibration is wrong.

Every capture file must be matched to the target that was in view and to exactly where the robot put it. Most of the procedure is about getting that table right.

The session at a glance



Preparation before the first session



Robot

- Check the documented absolute accuracy: 0.1 mm or better
- Weigh each fixture; enter its tool load data
- Write the robot program; note where it writes pose_log.csv
- Dry-run every pose at reduced speed
- Safety review of the new program



Sensor and computers

- Capture software set for the robot's trigger and the file names
- Exposure, gain and SGM parameters set and recorded: smoothing filters off, installation patch size
- 10 GB free per 1,000 frames
- Calibration software installed and self-tested



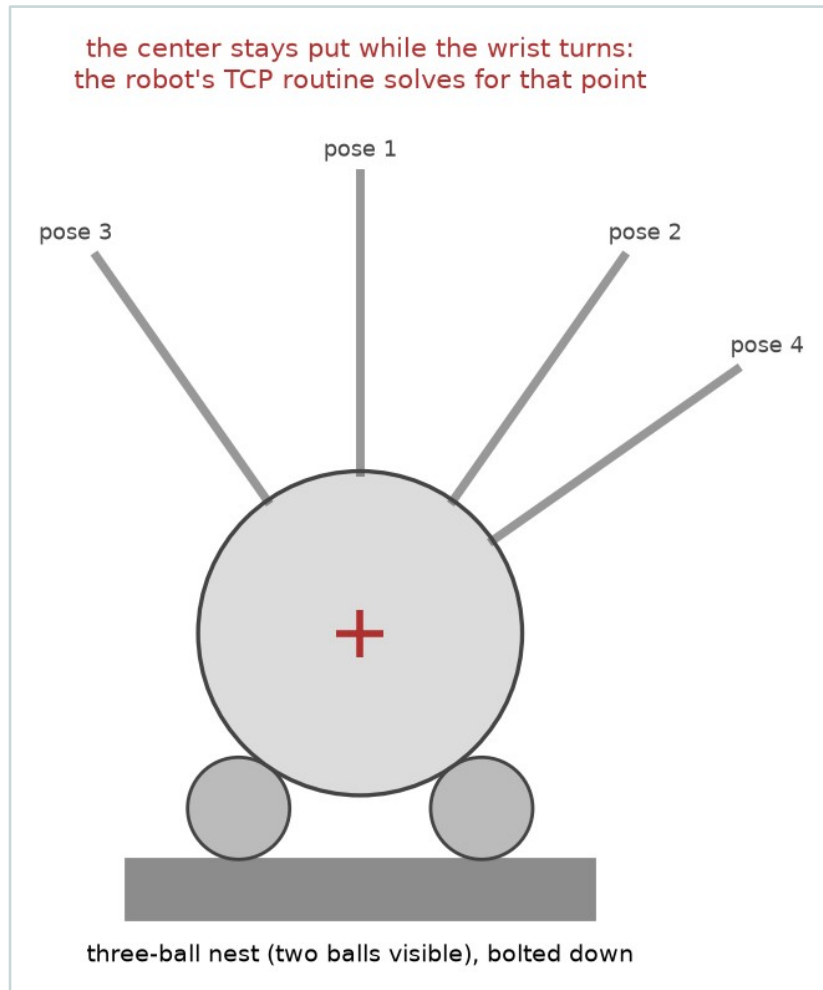
Cell

- Layout study: the 300 to 1,100 mm volume inside the robot's reach
- Table drilled and tapped for the nest and the run-out plate
- Robot base frame identified and recorded



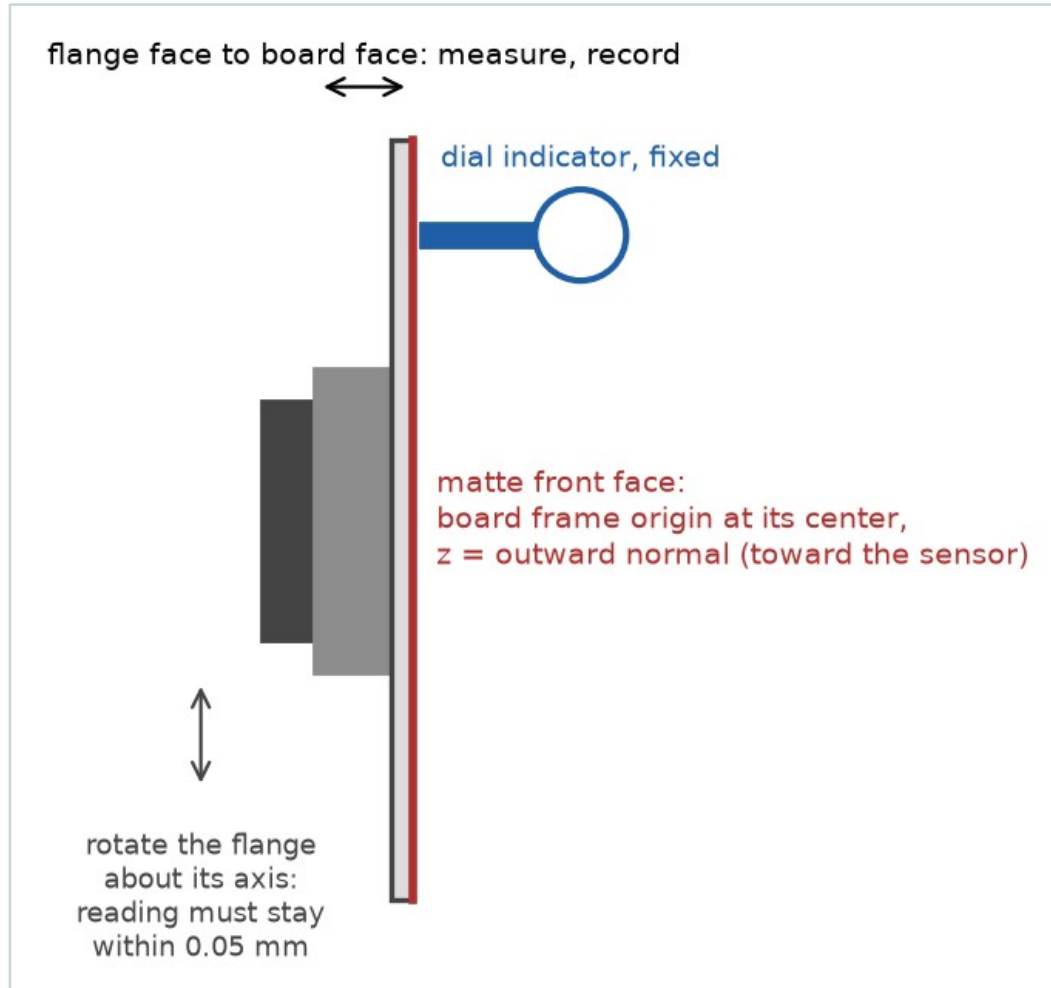
The fixtures and tools themselves are in the procurement and build deck, and in section 1 of the procedure.

The three-ball nest finds each sphere's center



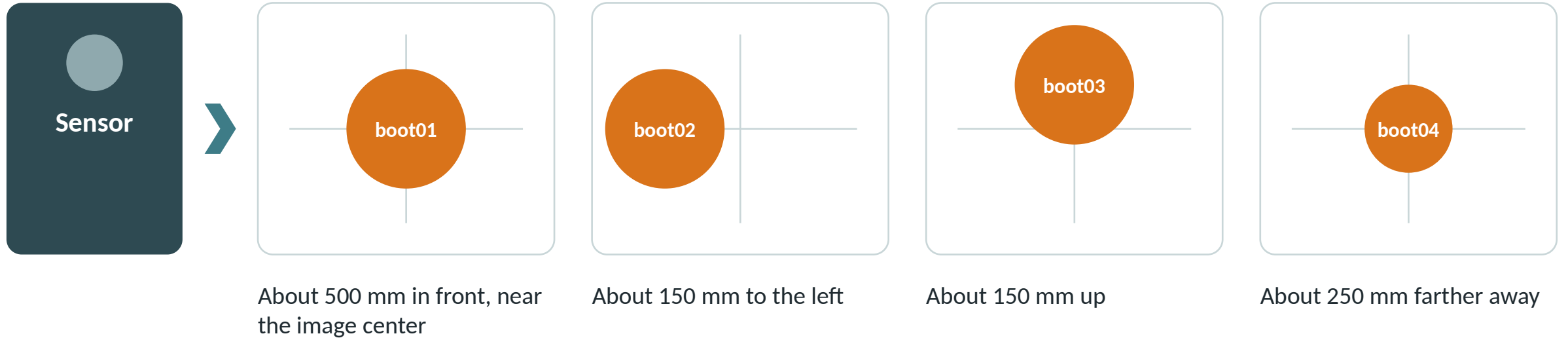
- 1 Enter a rough TCP from caliper measurements, and the fixture's tool load data
- 2 Run the robot's multi-orientation TCP routine with at least six seatings: stem up, tilted 40 degrees four ways, one twisted
- 3 Accept only a routine error of 0.1 mm or less
- 4 Re-seat at a very different wrist angle: positions agree within 0.1 mm
- 5 Save TCP_SPHERE_A_76mm and TCP_SPHERE_B_152mm

The board's tool frame sits on its front face



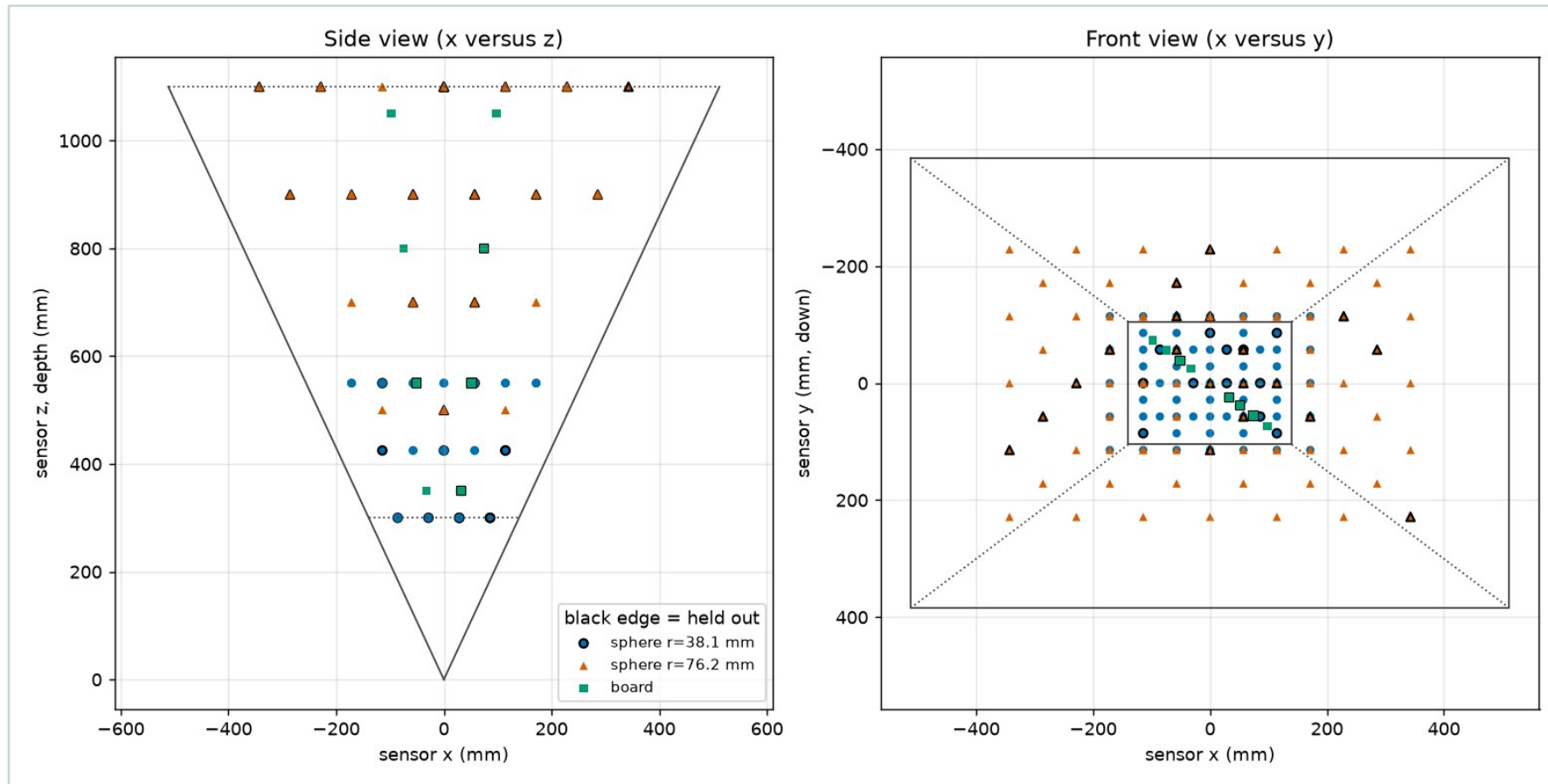
- ✓ Runout: rotate joint 6 through 360 degrees; the dial indicator stays within 0.05 mm
- ✓ Measure flange-to-face distance D at four places; they agree within 0.05 mm
- ✓ Tool frame: origin at the face center, z out of the face, x along the long edge
- ✓ A few degrees of error in x is harmless; an error in z is not
- ✓ Save as TOOL_BOARD with its load data; record D, the runout and the board size

Four captures locate the sensor well enough to plan



- Record the robot's reported TCP position for each
- The check tool fits the sphere centers; the plan tool fits the sensor position
- Residuals of a few millimeters are fine; tens of millimeters mean a copying error or a wrong TCP

The plan spreads 186 poses through the volume



About 20 percent of poses are held out to test the fit; they are captured like all the others.

90

sphere A poses at 300, 425 and 550 mm

57

sphere B poses from 500 to 1,100 mm

39

board poses at 4 depths, tilted 0, 20 and 40 degrees

What the robot program must do



Inputs

- poses.csv from the plan tool: base-frame position and orientation of every pose, as a matrix, a quaternion and a rotation vector
- Three tool frames:
TCP_SPHERE_A_76mm,
TCP_SPHERE_B_152mm and
TOOL_BOARD



Behavior

- Visit poses in plan order: sphere A, sphere B, board
- Pause for each fixture change and activate its tool frame
- Same approach every time; settle 1.5 seconds
- Never change the base frame during the session



Outputs

- Trigger five frames per pose through the communication software, named `<pose_id>_Index00` to `_Index04`
- Append the REPORTED TCP pose to pose_log.csv in the controller's own orientation form; note where the file is written



Once the robot model and controller are settled, the robot program can be written with Claude Code.

Every pose runs the same short robot loop



Poses are recorded in the file header or in a pose log

Option A: in the file header

- The capture software writes robotPose, a 4 x 4 matrix, tool to base
- No separate log needed beyond the target sizes
- Use it if the capture software supports it

Option B: pose log and manifest

- The robot program appends one CSV line per pose
- Orientation in the controller's own form: KUKA A, B, C or FANUC W, P, R, quaternion, rotation vector, matrix
- make_manifest matches files to poses and names every mismatch
- Recommended as a backup even with option A



s038_z0425_017, sphere, 38.10, , , 812.40, -33.21, 455.02, none, , , ,

Check the captures before the robot is put away



Low valid fraction

below 50 percent of frames

Target not read: check exposure, or the target was outside the field



Touches the image border

valid pixels at the edge

Target partly out of view: the pose is unusable; re-plan it slightly inward



Sphere center off

more than 2 mm

One pose: position copied wrongly. One sphere: wrong TCP. Growing across the volume: base frame or robot accuracy



Board normal off

more than 2 degrees

Board tool frame orientation wrong, or the rotation type in the log is misnamed

Re-capture a flagged pose under a new id, for example s038_z0425_017r; never delete lines from the log.

Two shape checks test the corrected surfaces



Sphere shape

- Each held-out sphere is fitted with its radius free
- After correction the fitted radius should match the measured radius
- Judge each error against its standard error, which is large for a small or distant sphere



Board flatness

- Each held-out board is fitted with its own plane
- After correction the spread should fall toward the plate's flatness
- Sees only errors that bend the board; mainly catches the fixed pattern



Neither check tests the scale of the volume or a smooth robot error; those rest on the robot's 0.1 mm accuracy.

Seven things that spoil a session



Moving or bumping the sensor: everything after it is a new session



Fingerprints or gloss: wipe the targets with isopropyl alcohol



Changing exposure, gain or the base frame mid-session



Letting the stem point toward the sensor: the sphere must hide it



Logging the commanded pose instead of the reported pose



Mixing up the two spheres' radii in the pose log



A loose adapter or a bent stem: roll the stem on a flat surface first

What to hand over at the end of the session

- ✓ captures/ with every .mc file, named <pose_id>_Index<nn>.mc
- ✓ pose_log.csv, or confirmation that the headers carry robotPose
- ✓ captures/manifest.csv, built without errors
- ✓ captures/check.json with no flags, or a note on each
- ✓ plan/ files as used, boot.json and the bootstrap captures
- ✓ session_notes.txt: sensor serial, warm-up, exposure, gain and SGM parameters, base frame, TCPs, board D and runout, sphere diameters, temperature, pose log location
- ✓ Photos (a phone camera is fine) of the sensor mount, each fixture on the flange, and the nest



Software

- plan_poses, make_manifest and check_captures in sphcal/cli
- Install and self-test: procedure appendix C
- Every tool prints its options with --help